

Comparison of neural network NARX and NARMAX models for multi-step prediction using simulated and experimental data

Joseph Kelley, Martin T. Hagan *

Oklahoma State University, Stillwater, OK, USA

ARTICLE INFO

Keywords:

Multi-step forecasting
Neural network
Recurrent neural network
NARX
NARMAX

ABSTRACT

This paper provides the first extensive comparison of NARX and NARMAX models for multi-step prediction. NARMAX models are more complex and require more sophisticated training procedures than NARX models, as described in this paper. However, for some types of data, the NARMAX models provide significantly improved forecasts for shorter prediction horizons, although the predictions converge as the horizon increases. This paper analyzes the NARX and NARMAX predictors in depth on simulated and experimental data to determine those situations where the NARMAX predictor is preferred.

1. Introduction

Predictions can be performed for single or multiple steps ahead. The multiple step ahead predictions are significantly more difficult. In part this is because errors tend to accumulate. In addition, model instabilities become critical. Multi-step predictions are crucial for such applications as model predictive control (see [Hagan et al. \(2002\)](#)).

Multi-step prediction could be thought of as a special case of nonlinear modeling. Examples of this type of modeling are found in [Jeong et al. \(2010\)](#), [Precup et al. \(2022\)](#), [Verma et al. \(2022\)](#), [Pozna et al. \(2010\)](#), [Ketsarapong et al. \(2012\)](#), and [Hedrea et al. \(2021\)](#). However, in this paper we focus only on multi-step prediction with two of the most popular models.

The multi-step models we use and compare in this paper are the Nonlinear AutoRegressive model with eXogenous inputs (NARX) and the Nonlinear Autoregressive Moving Average model with eXogenous inputs (NARMAX). The NARX model is more popular and is simpler to implement and train. It is important to understand those situations where the cost of implementing the NARMAX multi-step predictor is outweighed by its improved performance. This has not been addressed in previous work, and we focus on that topic in this paper.

The NARMAX model was first introduced in [Billings and Leontaritis \(1982\)](#) as a nonlinear version of the ARMAX model, although the less general NARX model is more commonly used. Often the NARX model is referred to in the literature as a NARMAX model (see [Narendra and Parthasarathy \(1990\)](#)). The difference between the models is that the NARMAX model includes delayed values of the prediction errors, whereas the NARX model includes only the current prediction error. This is important because the inclusion of delayed prediction errors

requires that dynamic backpropagation be used in order to compute the exact gradient. This also causes NARMAX training to be more difficult, as described later in this paper. The focus of this work is to determine if, and under what circumstances, the increased difficulty provides improved performance.

There are many variations of NARX and NARMAX models, due to the use of different forms of nonlinearity. The original implementation ([Billings & Leontaritis, 1982](#)) used polynomial nonlinearities, and this has continued to be popular ([Billings, 2013](#); [McHugh et al., 2018](#)). However, several other functions have also been used, such as radial basis functions ([Chen et al., 1990](#)), fuzzy systems ([Gao & Er, 2005](#), [Golob & Tovornik, 2008](#)) and wavelets ([Wei & Billings, 2006](#)). Using polynomials, radial basis functions or wavelets enable a model that is linear in the parameters. However, for this work we will use a multilayer neural network to implement the nonlinearity, as in [Narendra and Parthasarathy \(1990\)](#). Our objective here is to compare the NARX structure with the NARMAX structure as a multi-step predictor, rather than to compare various implementations of NARX and NARMAX models.

Most previous work on NARX/NARMAX models have focused on one-step predictions (see [Chen and Billings \(1989\)](#), [Gao and Er \(2005\)](#), [Billings \(2013\)](#)). This is especially true for NARMAX models. To the authors' knowledge, there has not been a direct comparison of NARX and NARMAX models for multi-step predictions. (In [Acuña et al. \(2012\)](#) NARX and NARMAX are compared, but they are trained to minimize one-step predictions.) Training NARMAX models for multi-step predictions requires the use of dynamic backpropagation for the exact calculation of gradients of multi-step errors ([De Jesus & Hagan, 2007](#)).

* Corresponding author.

E-mail addresses: joseph.kelley10@okstate.edu (J. Kelley), mhagan@okstate.edu (M.T. Hagan).

In addition, the error surface of such networks has been shown to have spurious valleys that inhibit efficient training (Phan & Hagan, 2013). In this paper we demonstrate that NARMAX models, when trained to minimize multi-step prediction errors using recent algorithms (see Jafari and Hagan (2018)), can provide improved multi-step predictions (when compared to NARX models), especially for shorter time horizons. We also provide a detailed analysis of why the NARMAX predictions are more accurate.

There are several novel aspects of this paper. It is the first extensive comparison between NARX and NARMAX models for multi-step prediction. It is the first application of a new recurrent network training procedure to NARMAX models. Also, it presents a new technique for loading tapped delay lines in NARMAX models for use in multi-step predictions. Finally, we provide new insights into the performance of multi-step NARMAX predictions.

We will begin this paper by reviewing linear ARX and ARMAX models for one-step and multi-step predictions (Section 2). These linear models are the building blocks for the nonlinear models and illustrate the importance of using the moving average of the prediction errors. After the linear models, we will present the nonlinear NARX and NARMAX models in Sections 3 and 4. The last Section 5 will provide a detailed analysis of performance differences between the NARX and NARMAX models for multi-step predictions on both simulated and experimental data.

2. Linear models

In this paper we take the view that the Data Generating Process (DGP) can be represented to a reasonable extent by a single model. A general linear model of a single-input single-output (SISO) system is shown in Eq. (1), where $y(t)$ is the endogenous, or output, variable, $u(t)$ is the exogenous, or input, variable, and $e(t)$ is the innovations process. The term $G(q) \cdot u(t)$ represents the part of $y(t)$ that can be predicted from (caused by) $u(t)$, and $H(q) \cdot e(t)$ represents stochastic disturbances and measurement noise that are uncorrelated with $u(t)$. $G(q)$ and $H(q)$ are transfer functions that must be identified, and q represents the time shift operator. It is assumed that the innovation sequence $e(t)$ is stationary white noise.

$$y(t) = G(q) \cdot u(t) + H(q) \cdot e(t) \quad (1)$$

When comparing ARX and ARMAX models (and later NARX and NARMAX models) the term $H(q) \cdot e(t)$ will be very important. Since ARMAX models have more flexibility in representing this term, they will be better able to model systems for which the DGP has complex disturbances or measurement noise.

2.1. ARX model

For an ARX model, shown in Eq. (2), the current output, $y(t)$, is a linear function of previous outputs, previous inputs $u(t)$ and the current innovation $e(t)$.

$$y(t) + a_1 y(t-1) + \dots + a_{n_y} y(t-n_y) = b_1 u(t-1) + \dots + b_{n_u} u(t-n_u) + e(t) \quad (2)$$

If we define

$$B(q) = b_1 \cdot q^{-1} + b_2 \cdot q^{-2} + \dots + b_{n_u} \cdot q^{-n_u} \quad (3)$$

$$A(q) = 1 + a_1 \cdot q^{-1} + a_2 \cdot q^{-2} + \dots + a_{n_y} \cdot q^{-n_y} \quad (4)$$

the ARX model can be written in the form of Eq. (1)

$$y(t) = \underbrace{\frac{B(q)}{A(q)}}_{G(q)} \cdot u(t) + \underbrace{\frac{1}{A(q)}}_{H(q)} \cdot e(t) \quad (5)$$

Note that a limitation of the ARX model is that $G(q)$ and $H(q)$ cannot be independently specified, since they share a common denominator, and the numerator of $H(q)$ is 1.

2.2. ARX predictions

For predictions, we use the minimum mean square error estimator, which is generally the conditional mean. For example, if we are predicting $y(t+k)$ based on observations up to time t , we would use

$$\begin{aligned} \hat{y}(t+k | t) &= \hat{y}_t(k) \\ &= E[y(t+k) | y(t), y(t-1), \dots] \\ &= E_t[y(t+k)] \end{aligned} \quad (6)$$

To find the ARX predictor, we solve Eq. (2) for $y(t+k)$ and then take the conditional expectation. Because the equation is linear, we can sum the expectations for the individual terms, which can be computed as follows (for $j \geq 0$).

$$E_t[y(t-j)] = y(t-j) \quad (7)$$

$$E_t[y(t+j)] = \hat{y}_t(j) \quad (8)$$

$$E_t[e(t-j)] = e(t-j) = y(t-j) - \hat{y}_{t-j-1}(1) \quad (9)$$

$$E_t[e(t+j)] = 0 \quad (10)$$

$$E_t[u(t+j)] = u(t+j) \quad (11)$$

(Note that the one-step prediction error is the model innovation $e(t)$ – see Ljung (1999).)

Applying these results to Eq. (2), we can obtain the k -step prediction as

$$\begin{aligned} \hat{y}_t(k) &= -a_1 \hat{y}_t(k-1) - \dots - a_{n_y} \hat{y}_t(k-n_y) \\ &\quad + b_1 u(t+k-1) + \dots + b_{n_u} u(t+k-n_u) \end{aligned} \quad (12)$$

where

$$\hat{y}_t(j) = y(t+j) \text{ if } j \leq 0 \quad (13)$$

If the innovations are assumed to be Gaussian, maximum likelihood methods would select the parameters to minimize the mean square one-step prediction errors. However, Cox (1961), Tiao and Xu (1993) and Tiao and Tsay (1994) suggest that minimizing multi-step errors provides more robust predictions. We have found this to be especially true for nonlinear extensions of ARX and ARMAX models, as we will discuss in later sections.

2.3. ARMAX model

The ARMAX model is shown in Eq. (14). The difference between the ARX and ARMAX is the moving average of innovations.

$$\begin{aligned} y(t) + a_1 y(t-1) + \dots + a_{n_y} y(t-n_y) = & b_1 u(t-1) + \dots + b_{n_u} u(t-n_u) + \\ & e(t) + c_1 e(t-1) + \dots + c_{n_e} e(t-n_e) \end{aligned} \quad (14)$$

If we define

$$C(q) = 1 + c_1 \cdot q^{-1} + c_2 \cdot q^{-2} + \dots + c_{n_e} \cdot q^{-n_e} \quad (15)$$

the ARMAX model can be written in the form of Eq. (1)

$$y(t) = \underbrace{\frac{B(q)}{A(q)}}_{G(q)} \cdot u(t) + \underbrace{\frac{C(q)}{A(q)}}_{H(q)} \cdot e(t) \quad (16)$$

The ARMAX model is more general than the ARX model, as the numerator of $H(q)$ can now be specified. $G(q)$ and $H(q)$ continue to share a denominator, although the appropriate choice of $C(q)$ could be used to mitigate this restriction. We expect that the increased flexibility of the ARMAX model can enable improved predictions with parsimonious models. This is especially true when the DGP has complex disturbances or measurement noise. It is not unreasonable to expect that this flexibility would carry over to the nonlinear NARMAX model, which is presented in a later section.

2.4. ARMAX predictions

For ARMAX predictions, using the same strategy as for the ARX predictions, we can find forms similar to Eq. (12), with the addition of terms due to the innovations, which are determined by Eq. (9) and (10). From Eq. (10) we know expectations of future innovations are zero. From Eq. (9) expectations of previous innovations are the one-step prediction errors. Therefore, if we are predicting n_e steps or more ahead, the moving average terms no longer contribute to the prediction. With each step that we move the prediction forward, another observation is replaced by a prediction, and another innovation is replaced by 0.

The ARMAX predictor equations are

$$\begin{aligned}\hat{y}_t(k) = & -a_1 \hat{y}_t(k-1) - \dots - a_{n_y} \hat{y}_t(k-n_y) \\ & + b_1 u(t+k-1) + \dots + b_{n_u} u(t+k-n_u) \\ & \hat{e}_t(k) + c_1 \hat{e}_t(k-1) + \dots + c_{n_e} \hat{e}_t(k-n_e)\end{aligned}\quad (17)$$

where

$$\hat{y}_t(j) = y(t+j) \text{ if } j \leq 0 \quad (18)$$

and

$$\begin{aligned}\hat{e}_t(j) = & E[e(t+j) | y(t), y(t-1), \dots] \\ = & \begin{cases} e(t+j) = y(t+j) - \hat{y}_{t+j-1}(1) & j \leq 0 \\ 0 & j > 0 \end{cases}\end{aligned}\quad (19)$$

3. NARX model

In the ARX model the next value of the output is a linear function of previous values of the input and output. For the nonlinear NARX model the linear function is replaced by a nonlinear function $f(\cdot)$, as in Eq. (20).

$$y(t) = f(y(t-1), y(t-2), \dots, y(t-n_y), u(t-1), u(t-2), \dots, u(t-n_u)) + e(t) \quad (20)$$

For this paper the nonlinear function $f(\cdot)$ will be implemented by a multilayer neural network.

Because of the nonlinearity in the NARX model, it is impractical to compute the conditional expectation needed for the minimum mean square error predictor. However, we will use the same form of predictor as the linear model in Eq. (12), where predictions replace observations for future time steps.

3.1. NARX one-step predictor

Fig. 1 shows the one-step predictor for the NARX model, if a single tanh hidden layer is used for the neural network implementation of $f(\cdot)$. (In general, the NARX model could use any number of layers in the neural network.) The TDL blocks are tapped delay lines that store previous values of the input. For example, if $u(t)$ is input to the TDL block, the output is a concatenation of $u(t-1), u(t-2), \dots, u(t-n_u)$. The outputs of the two TDLs in Fig. 1 represent the AR and X components of the model.

As in the ARX case, Eq. (12), when $k = 1$, the one-step predictor takes as input previous observations, and no previous predictions are required. From the point of view of training the neural network, this is convenient, because the standard (static) backpropagation algorithm can be used to compute the gradient.

The network weights and biases (W^u , W^y , W^2 , b^1 , and b^2) are optimized during training to minimize the sum square error between one-step predictions $\hat{y}_t(1)$ and measurements $y(t+1)$, as defined by the performance index $F(\mathbf{x})$ in Eq. (21). (The variables S^1 and S^2 in Fig. 1 represent the numbers of neurons in each layer.)

$$F(\mathbf{x}) = \sum_{t=n_0}^{Q-1} (y(t+1) - \hat{y}_t(1))^2 \quad (21)$$

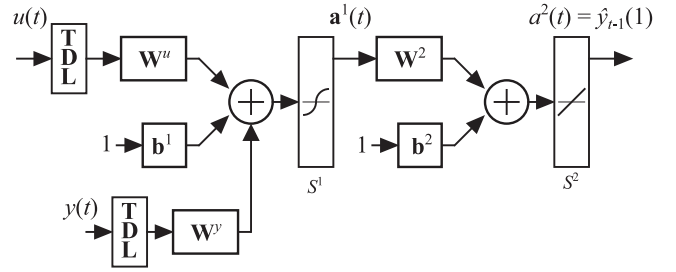


Fig. 1. NARX one-step predictor model.

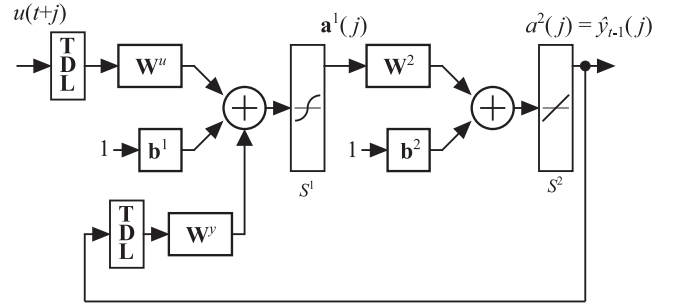


Fig. 2. NARX multi-step predictor model.

where $n_0 = \max(n_y, n_u)$, $\mathbf{x}$ is the vector containing all weights and biases, and the observations available for training are the inputs $\{u(1), u(2), \dots, u(Q)\}$ and the outputs $\{y(1), y(2), \dots, y(Q)\}$.

3.2. NARX multi-step predictor

To form the NARX multi-step predictor, we substitute predictions for observations in the one-step predictor, as was done in the ARX multi-step predictor of Eq. (12). This is the Iterated method for multi-step prediction, and is represented in Fig. 2.

When considering the one-step predictor of Fig. 1 and the multi-step predictor of Fig. 2, it is important to keep in mind that in Fig. 1 the time origin (t in Eq. (12)) is being incremented, but the prediction horizon (k in Eq. (12)) is fixed at $k = 1$. In Fig. 2 the reverse is true – $\hat{y}_{t-1}(j)$ is being pushed into the TDL as j is incremented, with t fixed.

3.2.1. Prediction horizon/TDL loading

In Fig. 2 the prediction horizon j increments with the number of cycles through the feedback loop. Initially, the output TDL is loaded with the first n_0 observations $\{y(1), y(2), \dots, y(n_0)\}$. For the first cycle the predictor produces $\hat{y}_{n_0}(1)$, which is then pushed into the output TDL queue, while $y(1)$ is popped out. On the next cycle, the predictor produces $\hat{y}_{n_0}(2)$. In each subsequent cycle the prediction horizon increases by 1. The prediction origin remains $t_0 = n_0$. To find the predictions at a new origin, the TDLs must be reloaded with the corresponding observations. (The TDL loading process becomes more complex and more critical with the NARMAX model, as we will discuss in the NARMAX section.)

As a practical matter, when training the multi-step predictor, we select a set of N prediction origins, $O = \{t_{01}, t_{02}, \dots, t_{0N}\}$, from the training set time series and a corresponding subsequence of observations for each origin. The first $n_0 = \max(n_y, n_u)$ observations from a subsequence are loaded into the TDL, and the remaining observations of the subsequence are run through the network. Each subsequence will have $n_0 + H$ observations, where H is the maximum prediction horizon, so that the network will produce predictions for time horizons 1 to H for each subsequence. When training the network, the subsequences can be chosen so that they overlap, but we found that this produces significant redundancy in the data set, so we divide the data into non-overlapping or randomly overlapping subsequences.

3.2.2. Multi-step prediction performance index

The weights and biases in the multi-step predictor could be the same as those in the one-step predictor, which were chosen to minimize one-step prediction error. However, as discussed previously, minimizing multi-step prediction errors produces predictions that are more robust. In addition, it has been shown that the weights obtained from the one-step predictor can produce unstable results when used for multi-step prediction, because of the feedback connection in the multi-step predictor model (see Jafari and Hagan (2018)).

The performance index for multi-step predictions is

$$F(\mathbf{x}) = \sum_{i \in O} \sum_{j=1}^H \left(y(t + n_0 + j) - \hat{y}_{t+n_0}(j) \right)^2 \quad (22)$$

The multi-step predictor of Fig. 2 is a recurrent neural network. As such, dynamic backpropagation must be used to obtain the exact gradient.

3.3. Training algorithm for multi-step predictions

When training the multi-step NARX prediction model there is an issue with stability because of the feedback loop. Since the one-step prediction model does not have feedback, weights can be initially trained for one-step predictions, and then the prediction horizon can be gradually increased during training. This approach is based on the work of Jafari and Hagan (2015) and is detailed below. The algorithm will continue to run until our desired prediction horizon is reached. We will train the NARX model using the Levenberg–Marquardt (LM) algorithm (see Hagan and Menhaj (1994)) shown in Eq. (23).

$$\mathbf{x}_{k+1} = \mathbf{x}_k - [\mathbf{J}^T(\mathbf{x}_k)\mathbf{J}(\mathbf{x}_k) + \mu_k \mathbf{I}]^{-1} \mathbf{J}^T(\mathbf{x}_k)\epsilon(\mathbf{x}_k) \quad (23)$$

where ϵ is the vector containing all prediction errors, and $\mathbf{J}(\mathbf{x}_k)$ is the Jacobian:

$$[\mathbf{J}]_{h,l} = \frac{\partial \epsilon_h}{\partial \mathbf{x}_l^T} \quad (24)$$

When using Eq. (23), the parameter μ is increased whenever $F(\mathbf{x})$ fails to decrease. If μ reaches μ_{max} , the algorithm generally stops. This can occur when the algorithm is in a very sharp valley in the error surface. The procedure below uses this idea to detect the spurious valleys of recurrent networks described in Phan and Hagan (2013).

1. Set the prediction horizon to 1, and train the one-step ahead (open loop) NARX model for a fixed number of iterations using the LM algorithm of Eq. (23).
2. Convert to the multi-step (closed loop) NARX model, as in Fig. 2. (Note that the weight matrices maintain the same values.)
3. With the weights fixed, compute the mean square error (MSE) for prediction horizons from 1 to *maxstep* steps beyond the current horizon. Find the local minima of MSE with respect to the prediction horizon. Select the local minimum with the lowest MSE as the next prediction horizon. (Whenever the prediction horizon is changed, new origins and subsequences must be selected.)
4. Train the multi-step NARX model with the new prediction horizon using the LM algorithm of Eq. (23). At each iteration of the LM algorithm, if $\mu > \mu_{max}$, remove the subsequence that has the largest gradient and restart the training. If $F(\mathbf{x})$ of Eq. (22) does not decrease, continue removing the subsequence with the next largest gradient until $F(\mathbf{x})$ decreases. Once $F(\mathbf{x})$ decreases, return all subsequences. Continue for a fixed number of iterations. If the maximum horizon has been reached, stop, otherwise return to Step 3.

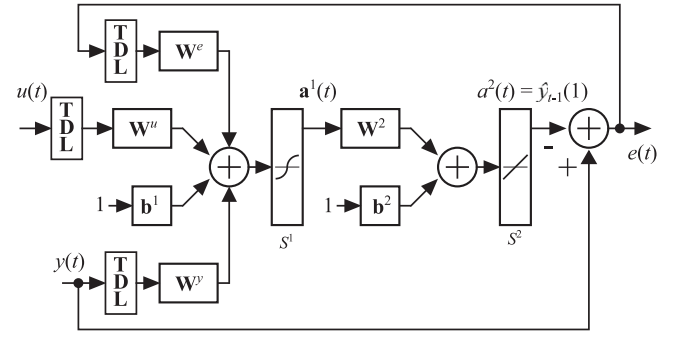


Fig. 3. NARMAX one-step predictor.

4. NARMAX model

Following the development of the NARX model, the NARMAX model takes the ARMAX model and replaces the linear function of inputs and replaces it with a nonlinear function, as in Eq. (25).

$$y(t) = f(y(t-1), y(t-2), \dots, y(t-n_y), \\ u(t-1), u(t-2), \dots, u(t-n_u), \\ e(t-1), e(t-2), \dots, e(t-n_e)) + e(t) \quad (25)$$

As with the NARX model, the nonlinearity makes it impractical to compute the exact conditional mean predictor for NARMAX. However, as in the NARX case, we use the same iterated predictor form as in the linear case. Also, as with the NARX model, it is useful to consider separately the one-step and multi-step predictor forms.

4.1. NARMAX one-step predictor

The NARMAX model for one-step prediction is shown in Fig. 3. The time origin t is being incremented here. There are now three TDLs, for the AR, X and MA components of the model. Because of the feedback loop in this model, dynamic backpropagation must be used to compute the exact gradient or Jacobian.

4.1.1. TDL loading

As with the NARX one-step predictor, the AR and X TDLs are loaded with the first n_0 observations of $y(t)$ and $u(t)$. The NARMAX one-step predictor also has the additional MA TDL. Unfortunately, the previous $e(t)$ values are not observable, so the TDL is initially loaded with zeros, which is the expected innovation. The initial zeros in the MA TDL will have a transient effect on the NARMAX calculations, which will settle after several time steps. We will indicate this transient length as L_t .

4.1.2. One-step prediction performance index

The performance index for the one-step NARMAX model is the same as for the one-step NARX model — Eq. (21). However, the original observation sequence is generally divided into smaller subsequences, so that there are several sets of sums. The reason is that, because of the feedback loop, there are potential instabilities that could occur. By shortening the length of the subsequence, we can prevent the instabilities from causing practical issues. The subsequence lengths are long enough to allow the transient effect of initial zeros in the MA TDL to settle, but short enough to prevent any instabilities from becoming manifest.

4.2. NARMAX multi-step predictor

The multi-step NARMAX predictor uses the same technique as the ARMAX model Eq. (17). Predictions will substitute for future observations, and future errors will be replaced by 0. The resulting predictor is shown in Fig. 4, where $\hat{e}_t(j)$ is defined in Eq. (19). As in the multi-step NARX model, the prediction horizon j is being incremented here, while the time horizon t is fixed.

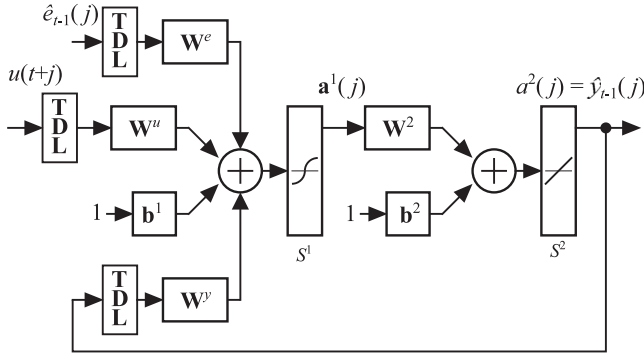


Fig. 4. NARMAX multi-step predictor.

4.2.1. Prediction horizon/TDL loading

The loading of the AR and X TDLs is the same as NARX. The MA TDL should contain either previous innovations, which are one-step prediction errors, or zeros for future time points, as described in Eq. (19). Therefore, to perform multi-step predictions, we first train the one-step predictor (Fig. 3) and compute the one-step prediction errors. These are used to load the MA TDL to begin the multi-step training. As the prediction horizon j is incremented in Fig. 4, zeros will be pushed into the MA TDL, as expectations of future innovations will be zero. Note that if $j > n_e$ the MA TDL will be filled with zeros, and the predictor will be equivalent to a NARX.

Whenever the prediction horizon is increased during the training process, the observation sequence is resegmented, as new origins and subsequences are selected. For the NARX model the subsequence length was $n_0 + H$. For the NARMAX model the subsequence length will be $2n_0 + H + L_t$. After a change in prediction horizon, the new subsequences are input to the one-step NARMAX predictor. The one-step predictor TDLs are initialized with the first n_0 steps of the subsequence, except the MA TDL, which is set to zeros. The predictor is run forward for the next $L_t + n_0$ steps. At that point, the TDLs will be appropriately loaded with the correct values for multi-step predictions, and the network connections are rearranged to the multi-step predictor format, leaving the weights and TDLs in place. The multi-step predictor can then calculate $\hat{y}_{2n_0+L_t}(j)$ for j from 1 to H .

4.2.2. Multi-step prediction performance index

The multi-step performance index for the NARMAX model is the same one used in the NARX case Eq. (22), except that $n_0 = \max(n_y, n_u, n_e)$.

4.3. Training algorithm for multi-step predictions

The NARMAX training algorithm for multi-step predictions is very similar to the NARX multi-step algorithm described earlier. This method has never before been applied to NARMAX models. The key difference is the requirement to fill the MA TDL. During each cycle through steps 3 and 4, the one-step prediction errors are recalculated, so that they can be loaded into the MA TDL for multi-step training. To calculate the one-step prediction errors, the multi-step predictor is converted back to the one-step predictor. (The transition between one-step and multi-step involves changing connections only — the weights maintain their values.)

5. Results

The objective of our work is to compare NARX and NARMAX multi-step predictors and to determine if NARMAX predictors are better in certain situations, and, if so, what those situations might be. To do that we performed two types of experiments. In the first experiments

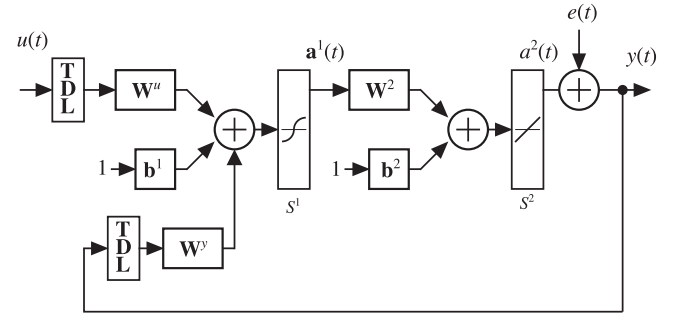


Fig. 5. NARX DGP.

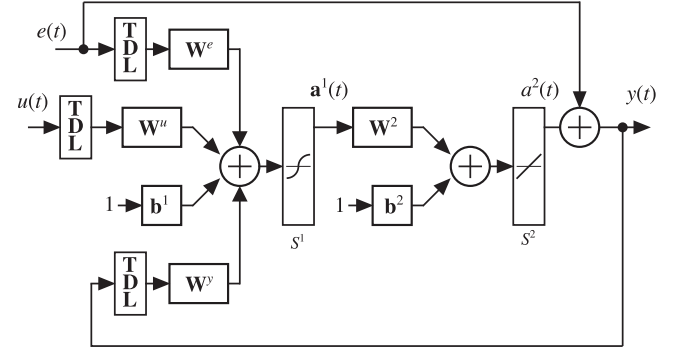


Fig. 6. NARMAX DGP.

we generated simulated data using random NARX and NARMAX DGPs and then fit NARX and NARMAX models to both data sets. These experiments demonstrate that the effectiveness of the two types of models depends on the DGP. In the second set of experiments we collect data from two different physical systems and fit both NARX and NARMAX models to both sets of data. The characteristics of the two data sets are different, and they demonstrate when the NARMAX model can be more effective for multi-step prediction than the NARX model.

5.1. Simulation results

We generated two simulated data sets using random NARX and NARMAX DGPs. We then fit both NARX and NARMAX models to both sets and compared the multistep predictions for both models.

The NARX DGP was implemented using the structure shown in Fig. 5, and the NARMAX DGP was implemented using the structure shown in Fig. 6. The weights of both DGPs were randomly set in the interval $[-0.4, 0.4]$. The exogenous input was a random skyline function (see Jafari and Hagan (2018)) with width in the interval $[10, 250]$ and height in the interval $[-0.8, 0.8]$. The innovation $e(t)$ was white gaussian noise with mean of 0 and variance of $2e-4$. All of the TDLs for the NARX and NARMAX DGPs were length 4, and there were 5 neurons in the hidden layers. Each simulated dataset contained 10,000 time points.

Both NARX and NARMAX models were trained on both datasets, using the training methods described earlier, to minimize mean square prediction error over a 20 time-step horizon. Because the maximum number of parameters in any of the models (161) was several orders of magnitude smaller than the dataset size (10,000), there was no chance of overfitting. In this way our study could concentrate on the difference in NARX and NARMAX structures and their effect on multi-step prediction accuracy. In addition, we maintained 5 neurons in the hidden layers (matching the DGP), and only varied the length of the TDL, since this is the essential difference between NARX and NARMAX models.

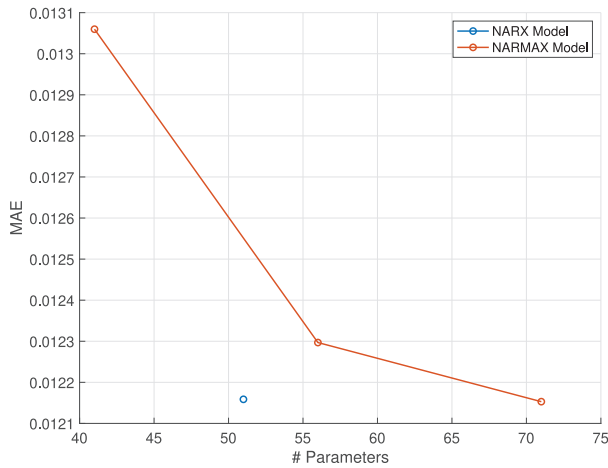


Fig. 7. Models trained on simulated NARX DGP with 5 neurons and 4 TDL — Plotted MAE for trained NARX models with 5 neurons, and TDL size of 4 — Plotted MAE for trained NARMAX models with 5 neurons, and TDL sizes 2, 3, and 4.

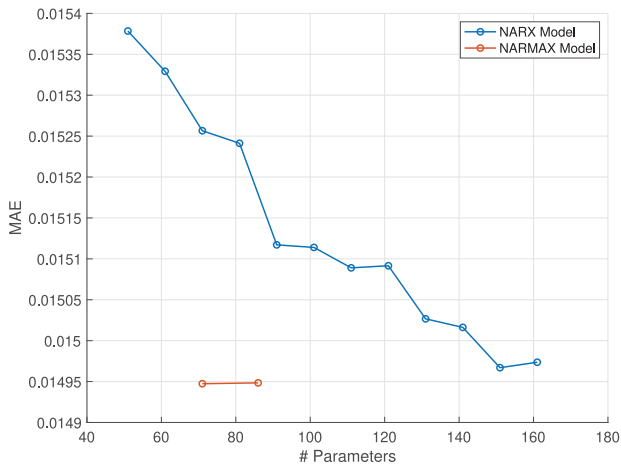


Fig. 8. Models trained on simulated NARMAX DGP with 5 neurons and 4 TDL — Plotted MAE for trained NARX models with 5 neurons, and TDL sizes of 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, and 15 — Plotted MAE for trained NARMAX models with 5 neurons, and TDL sizes 4 and 5.

Using the data from the NARX DGP, we first trained a NARX model with a TDL of 4 (which matched the DGP architecture) and then compared its MAE (Mean Absolute Error) over an horizon of 20 time steps with the MAE of NARMAX models with TDLs of lengths 2, 3 and 4. The trained NARMAX model performed worse than the trained NARX model with the same number of parameters (51). When the NARMAX and NARX models both had TDLs of length 4, they had a similar MAE, although the NARMAX model had 71 parameters. (10 Monte Carlo trials were run, and average MAE was used to compare models.) The results are illustrated in Fig. 7.

Using data from the NARMAX DGP, we first trained a NARMAX model with a TDL of 4 (matching the DGP) and then compared its MAE prediction errors over an horizon of 20 time steps with the errors of NARX models with TDLs with lengths from 4 to 15. The NARX model required a TDL of length 15 (161 parameters) to produce a similar MAE (over 20 prediction steps) as a NARMAX model with TDL of length 4 (71 parameters). The results are shown in Fig. 8.

The bottom line is that NARMAX models were at least as accurate as NARX models for both data sets when the TDLs were the same size. For NARX data, an accurate NARX model had fewer parameters than the equivalently accurate NARMAX model, and, for NARMAX data, an

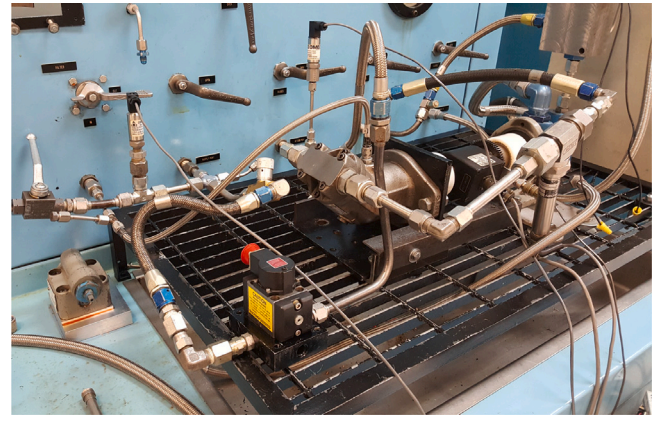


Fig. 9. Hydraulic servo-motor.

accurate NARMAX model had fewer parameters than an equivalently accurate NARX model.

In the next subsection we are going to explore the differences between NARX and NARMAX predictors in more detail on two experimental datasets. These datasets have been chosen to illustrate how the relative accuracies of NARX and NARMAX multi-step predictors are heavily dependent on the DGP. In one dataset the DGP has NARX characteristics, and in the other dataset the DGP has NARMAX characteristics.

5.2. Experimental results

In this subsection we compare the performances of NARX and NARMAX multi-step predictors using experimental data collected from a hydraulic servo-motor and a magnetic levitation system. (The multi-step predictors for these systems could be used as part of model predictive controllers, as described in Hagan et al. (2002), although that is not the focus of this work.)

The previous subsection demonstrated that the relative performances of NARX and NARMAX predictors will depend on the type of DGP. The two mechanical systems presented in this section are also representative of different types of DGP and will allow us to investigate the relative strengths and weaknesses of NARX and NARMAX predictors. The two systems are described below.

1. Hydraulic Servo-Motor:

System Description: A hydraulic servo-motor is used to control the output angular velocity of a motor coupled to a gearbox by controlling the input voltage into a servo-valve. (See Fig. 9.)

Data Description: The input into the servo-motor is voltage and the output is angular velocity of the gearbox. The voltage range is between 0 and 4 V. The data was collected for 100,000 time points at a sampling rate of 200 Hz. A more complete description is given in Kelley and Hagan (2020).

2. Magnetic Levitation (maglev):

System Description: A magnetic levitation system suspends a magnet above an electromagnet. The magnet is contained so that it can only move in the vertical direction. This is common in maglev trains. The data used here came from a stand-alone experimental system described in Jafari and Hagan (2018). (See Fig. 10.)

Data Description: The input into the system is voltage and the output is magnet position measured from an infrared sensor that has been filtered. The data was collected for 100,000 time points at a sampling rate of 50 Hz.

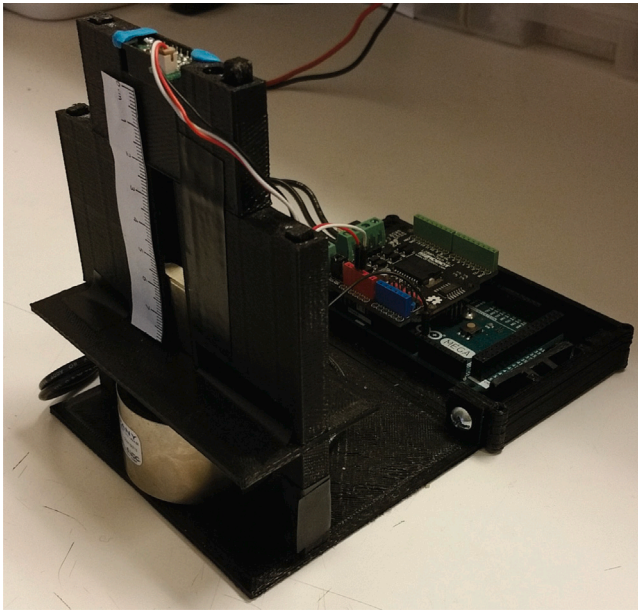


Fig. 10. Maglev system.

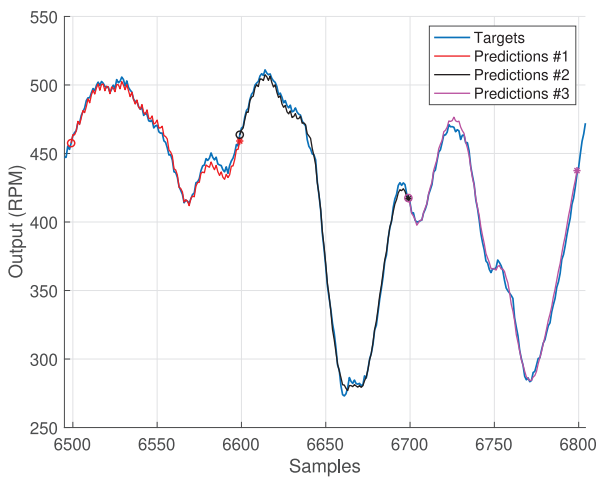


Fig. 11. NARX model trained on servo-motor data set for maximum horizon of 100 — 3 starting points for 100-step predictions. Circle marks the start of the prediction and star marks the end of the prediction.

The servo-motor and maglev systems are different in significant ways that will enable us to compare NARX and NARMAX predictions. The maglev system had accurate infrared sensors, and it had few disturbances, since it consisted of a simple magnet sliding along a rail. On the other hand, the servo-motor system had a significant amount of complex disturbances: (1) An electric motor ran the displacement pump; because it was mounted to the test stand, this lead to vibrations in the system; (2) A gear pump was used for the fixed displacement pump, and these pumps produce an oscillation around the nominal flow since the diameter of the outlet varies as the gears turn; (3) The hydraulic controlled servomotor was mounted to the test stand, which also lead to vibrations; (4) Finally, the system load was another hydraulic motor connected to the gearbox, which created additional vibrations.

The measurement noise and disturbances for the maglev were small and simple. The measurement noise and disturbances for the servo-motor were large and complex. Based on earlier discussions, we expect that the NARMAX model should provide significant benefits when

Table 1

NARX model architectures for servo-motor data set.

	Model 1	Model 2
Number of trained models	15	15
Output (AR) TDL size	8	15
Input (X) TDL size	8	15
Number of neurons	10	10

Table 2

NARMAX model architectures for servo-motor data set.

	Model 1	Model 2
Number of trained models	15	15
Output (AR) TDL size	8	15
Input (X) TDL size	8	15
Error (MA) TDL size	8	15
Number of neurons	10	10

Table 3

NARX model architectures for magnetic levitation data set.

	Model 1	Model 2
Number of trained models	15	15
Output (AR) TDL size	2	6
Input (X) TDL size	2	6
Number of neurons	10	10

Table 4

NARMAX model architectures for magnetic levitation data set.

	Model 1	Model 2
Number of trained models	15	15
Output (AR) TDL size	2	6
Input (X) TDL size	2	6
Error (MA) TDL size	2	6
Number of neurons	10	10

applied to the servo-motor, while the NARX model may be adequate for the maglev. We test these hypotheses in the following sections.

We trained NARX and NARMAX models with different TDL sizes for both servo-motor and maglev data sets using the algorithm described in Sections 3.3 and 4.3. For each prediction horizon we trained for 4000 iterations using the LM algorithm. Each dataset contained 100,000 time points, and the NARX and NARMAX models had a maximum of only 471 parameters, therefore there was no possibility of overfitting.

When selecting the network architectures we tested up to 20 neurons in the hidden layers. We found that using more than 10 neurons provided no additional reduction in prediction error. Therefore we used 10 hidden-layer neurons in all models and varied only the length of the TDLs, which is the main feature distinguishing NARX and NARMAX models.

For each network architecture we trained 15 different models. Each of the models was initialized with random weights and biases and then trained for a maximum prediction horizon (H) of 100. The model architectures for the servo motor data set are shown in Tables 1 and 2. The model architectures for the maglev data set are shown in Tables 3 and 4.

Figs. 11 and 12 show the ability of trained NARX models to predict multiple steps ahead for the servo-motor and maglev data sets. Both figures show 3 separate sets of predictions from 1 to 100 steps ahead.

To summarize model performance, we calculate the MAE over prediction horizons from 1 to 20. There are 10,000 randomly selected subsequences of 20-step predictions for each data set. Fig. 13 compares the MAE of NARX and NARMAX models over the 20 step horizon for the servo-motor data set. We can see that the NARMAX model achieves a lower MAE than the NARX model with fewer parameters. This suggests that the servo-motor can be represented by a NARMAX-like DGP. We will investigate this further later in this subsection.

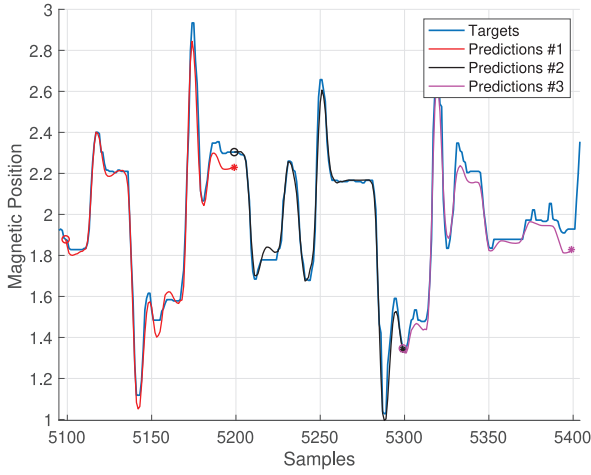


Fig. 12. NARX model trained on magnetic-levitation data set for maximum horizon of 100 — 3 starting points for 100-step predictions. Circle marks the start of the prediction and star marks the end of the prediction.

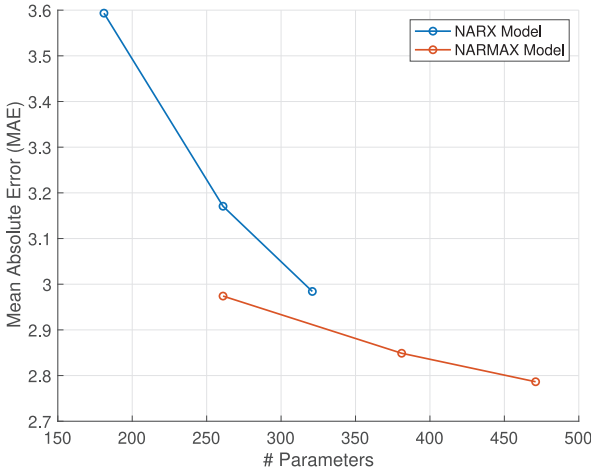


Fig. 13. Models trained on servo-motor data set — Plotted MAE averaged over 1 to 20 step predictions for trained NARX models with 10 neurons and TDL sizes of 8, 12, and 15 — Plotted MAE at 1 to 20 step predictions for trained NARMAX models with 10 neurons, and TDL sizes 8, 12, and 15.

Fig. 14 compares the MAE of NARX and NARMAX models for the maglev data set. It shows average MAE over prediction horizons from 1 to 20 and over the 15 models trained for each architecture. In this case, the NARX models produce lower MAE than the NARMAX models when both models have the same number of parameters. This suggests that the maglev can be best represented by a NARX-like DGP.

To investigate the difference between NARX and NARMAX predictors in more detail, we can consider how the prediction errors vary throughout the prediction horizon. The MAE for prediction horizons from 1 to 100 is shown in Fig. 15 for the servo-motor data set and Fig. 16 for the maglev data set. The thin lines are the MAE values for each of the 15 trained models. The thick lines represent the average MAE.

From these plots it becomes clear that the main difference between the NARX and NARMAX predictions occurs for short prediction horizons. For prediction horizons greater than 20 for the servo-motor, and greater than 10 for the maglev, the NARX and NARMAX have similar MAE values, which was expected since the NARMAX becomes a NARX for prediction horizons greater than the length of the MA TDL.

The NARMAX model, if correctly trained, should always produce predictions that are just as accurate as NARX models, if the size of the

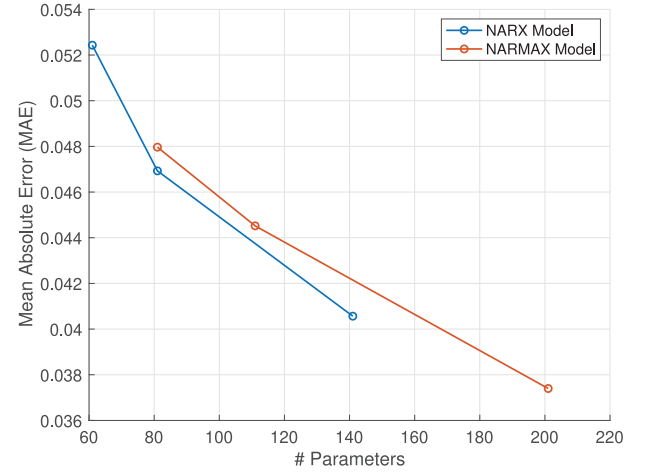


Fig. 14. Models trained on mag-lev data set — Plotted MAE at 7 step prediction for trained NARX models with 10 neurons, and TDL sizes of 2, 3, and 6 — Plotted MAE at 7 step prediction for trained NARMAX models with 10 neurons, and TDL sizes 2, 3, and 6.

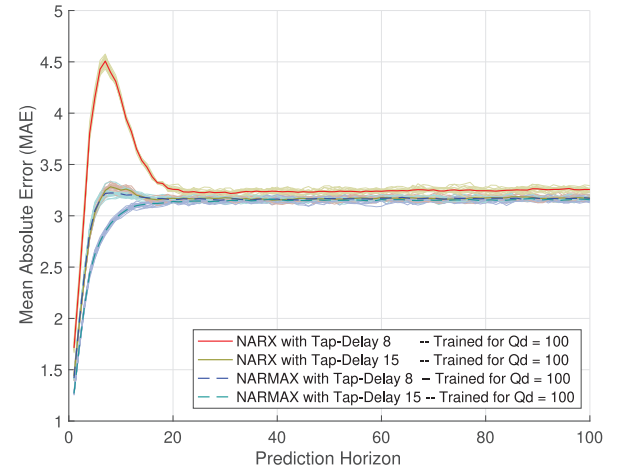


Fig. 15. NARX & NARMAX — Trained on servo-motor data set for multi-step prediction for a maximum horizon of 100.

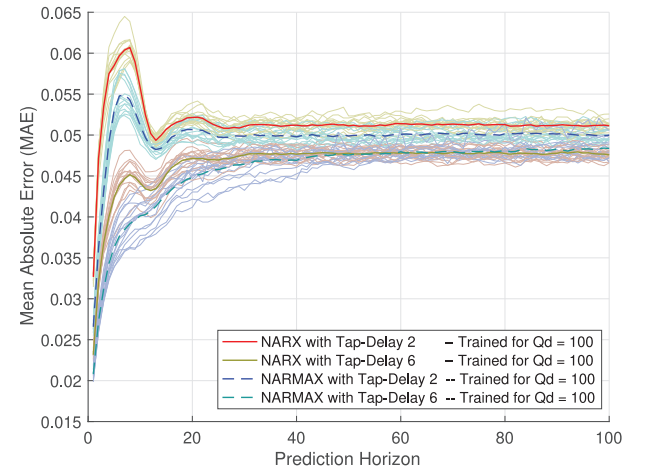


Fig. 16. NARX & NARMAX — Trained on magnetic levitation data set for multi-step prediction for a maximum horizon of 100.

TDLs is the same. This is because the NARX model is a subset of the NARMAX model. When the TDLs are the same size, a NARMAX model

Table 5
NARX vs NARMAX comparison for servo-motor data set.

Model	NARX	NARMAX
TDL	12	8
Neurons	10	10
Params	261	261
MAE-7	3.66	3.21
MAE-7 STD	0.033	0.019
MAE-40	3.18	3.17
MAE-40 STD	0.022	0.019

Table 6
NARX vs NARMAX comparison for maglev data set.

Model	NARX	NARMAX
TDL	3	2
Neurons	10	10
Params	81	81
MAE-7	0.0513	0.0538
MAE-7 STD	0.0013	0.0017
MAE-40	0.0479	0.0498
MAE-40 STD	0.0007	0.0009

can be made equivalent to any NARX model by setting the W^e matrix in the NARMAX model to $\mathbf{0}$, and making the other weights and biases in the NARMAX model equal to the NARX model weights.

However, if a DGP is truly NARX, it may require a NARMAX model with more parameters to achieve an accuracy better than the simplest fitting NARX model, as was demonstrated earlier with the simulated data. This can also be shown with the experimental data. For the servo-motor and maglev data sets, we took the best fitting NARMAX model and compared the results with the NARX model with the same number of parameters. We compared the models at two different prediction horizons. From Figs. 15 and 16 we see that the prediction errors differ most around 7 steps ahead, and differ least after 30 to 40 steps. As mentioned earlier, if the prediction horizon is greater than n_e , the MA TDL will be filled with zeros, and the NARMAX predictor will be equivalent to a NARX, so we expect the main differences will occur for shorter time horizons.

The results are shown in Tables 5 and 6. These show the MAE for prediction horizons of 7 and 40 averaged over 15 Monte Carlo trials, along with the corresponding standard deviations (STD). For the servo-motor data, which had significantly larger disturbances, the NARMAX model with the same number of parameters produced more accurate results than the NARX model at the 7 step prediction horizon, but the accuracies were similar at 40 steps. (A t-test for difference in mean MAE at 7 steps indicated a significant difference, with p -value of $3.55e-27$. A t-test at 40 steps did not indicate a significant difference, with p -value of 0.25.)

For the maglev data, at 7 steps, the NARX model produced more accurate results than the NARMAX model with the same number of parameters, but at 40 steps the accuracies were similar. (A t-test for difference in mean MAE at 7 steps indicated a significant difference, with p -value of $1.23e-4$. A t-test at 40 steps also indicated a significant difference, with p -value of $6.94e-7$.) When the TDL sizes were the same, the NARMAX model was at least as good for all data sets.

Overall, these results indicate that the DGP for the servo data is more closely related to a NARMAX form, and the DGP for the maglev data is more closely related to a NARX form. This matches with the results shown in Figs. 13 and 14. In addition, these experimental tests have shown that the main difference between NARX and NARMAX predictors occurs for short prediction horizons. For long predictions the two predictors are similar.

5.3. Characteristics of the nonlinear models

There is an interesting feature of Figs. 15 and 16 that might provide some insight into why the NARMAX models perform better in some

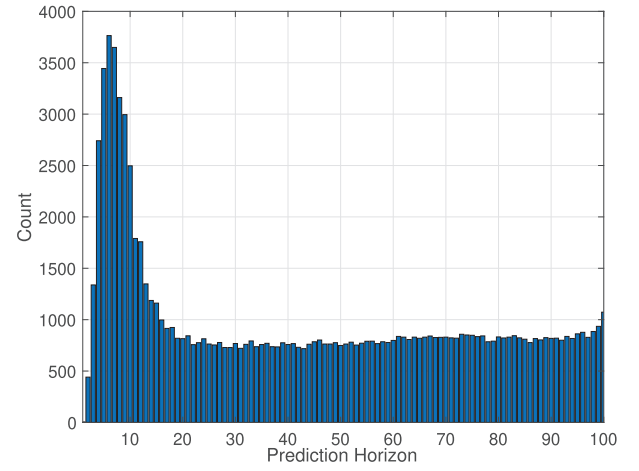


Fig. 17. NARX Model 1 – Histogram for maximum error location in a prediction horizon of 100 – Histogram created from 10,000 batches of 100-step samples – Trained on servo-motor data set for multi-step prediction for a maximum horizon of 100.

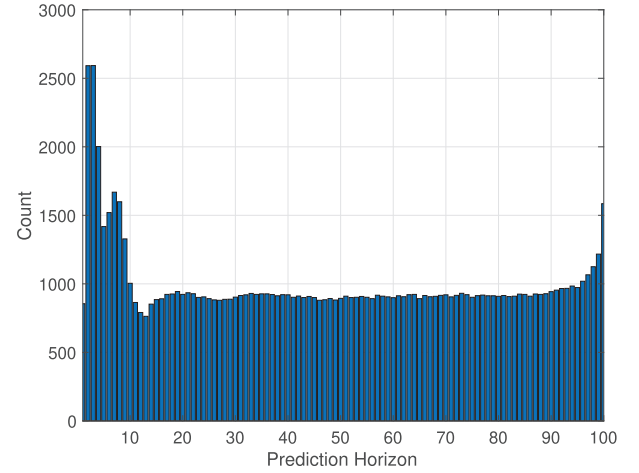


Fig. 18. NARX Model 1 – Histogram for maximum error location in a prediction horizon of 100 – Histogram created from 10,000 batches of 100-step samples – Trained on magnetic levitation data set for multi-step prediction for a maximum horizon of 100.

cases. We would expect that errors would increase monotonically with the prediction horizon — e.g., it should be easier to predict 7 steps than to predict 100 steps. However, for the NARX model with 8 tapped delays, the prediction errors for the servo motor are much larger for an horizon of 7 steps than for an horizon of 100 steps. A similar pattern occurs for the NARX model with 2 tapped delays for the maglev data. On the other hand, for the NARMAX models, we find the expected monotonic increase in errors with prediction horizon.

To investigate this phenomenon more closely, we considered individual predictions. Figs. 15 and 16 represent average errors at each prediction horizon. Figs. 17 and 18 are histograms that indicate, for each prediction origin, where the largest absolute error occurred. Fig. 17 indicates that for any given origin, the largest prediction error is much more likely to occur at 7 steps than at any horizon greater than 20. This is consistent with Fig. 15, where the largest MAE occurs at 7 steps.

For the maglev NARX Model 1 histogram in Fig. 18 there are two peaks — a large one at 2–3 steps and a second smaller one at 7 steps. The peak at 2–3 steps does not correlate with the MAE plot of Fig. 16. To investigate this further, Fig. 19 shows histograms of the maximum absolute prediction errors that occur at 2, 3 and 7 steps. From this figure, we can see that maximum errors occur more often at 2–3 steps, rather than 7 steps, which is consistent with Fig. 18. Why is there no peak in the MAE plot of Fig. 16 at 2–3 steps?

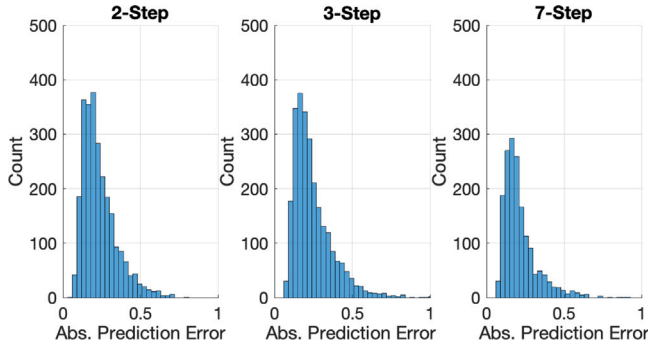


Fig. 19. NARX Model 1 – Histogram for the maximum absolute prediction error when located at 2, 3, 7-step predictions – Histogram created from 100,000 batches of 100-step samples – Trained on magnetic levitation data set for multi-step prediction for a maximum horizon of 100.

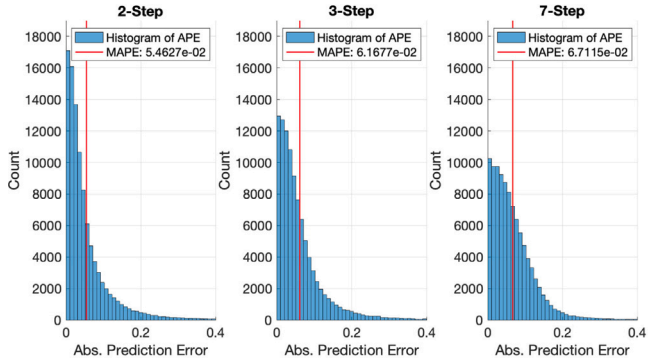


Fig. 20. NARX Model 1 – Histogram for absolute prediction error at 2, 3, 7-step predictions – Histogram created from 100,000 batches of 100-step samples – Trained on magnetic levitation data set for multi-step prediction for a maximum horizon of 100.

Fig. 20 shows histograms of all of the absolute prediction errors (APEs) at 2, 3 and 7 steps. It demonstrates that, although the peak error is more likely to occur at 2–3 steps, the peak error is larger when it occurs at 7 steps, so the MAE is larger at 7 steps.

We can gain further insight into the differences between NARX and NARMAX predictions from the 2D histograms (heatmaps) in Figs. 21 and 22. To create these figures, for each of the 10,000 subsequences with prediction horizons from 1 to 100, we found the horizon at which the maximum APE occurred and saved the value of the APE at that point. The heatmaps therefore indicate the horizons at which the maximum error occurred, and the magnitude of that error.

In Fig. 21, we can see that, for the servo-motor, the maximum error for the NARX model most often occurred when the prediction horizon was between 2 and 10. In addition, those errors were larger than when the maximum error occurred at larger horizons. On the other hand, Fig. 22 shows that the maximum error locations for the NARMAX model were almost equally distributed from 1 to 100 steps, and the magnitudes of the errors are fairly consistent across all horizons. This indicates that the NARMAX model more fully captured the complete dynamics of the DGP, while the NARX model had some difficulty with short-term (higher frequency) dynamics. These dynamics in the servo-motor were associated with system disturbances, analogous to the $H(q)e(t)$ term in the linear model (1). (We know that the ARMAX model is more flexible than the ARX model in representing this term.) We will show an example in the next subsection that illustrate this.

5.4. Prediction error performance for nonlinear models

In this section we will provide an example that illustrates some of the differences between NARX and NARMAX predictions. This example

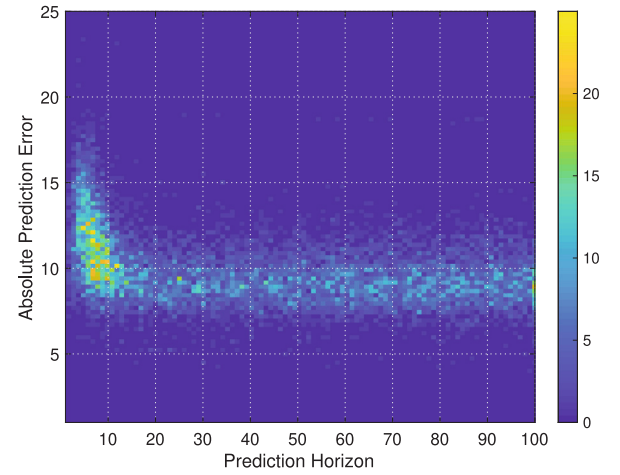


Fig. 21. NARX Model 1 – 2D histogram heat-map for maximum error value and location in a prediction horizon of 100 – Trained on servo-motor data set for multi-step prediction for a maximum horizon of 100.

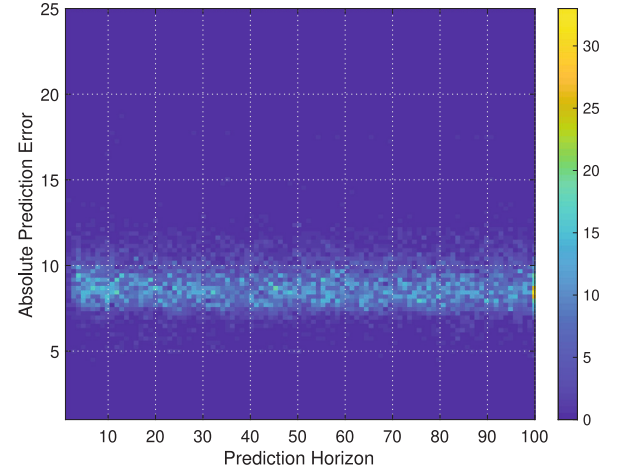


Fig. 22. NARMAX Model 1 – 2D histogram heat-map for maximum error value and location in a prediction horizon of 100 – Trained on servo-motor data set for multi-step prediction for a maximum horizon of 100.

was picked to illustrate how a NARX model can behave when the TDL size is not large enough to capture completely the system dynamics.

Fig. 23 shows 100-step predictions for both NARX and NARMAX model 1. There is a noticeable difference between the targets and NARX Model 1 predictions at 5–12 steps (data samples 6028–6035), which corresponds with the large MAE values in Fig. 15. The NARMAX Model 1 predictions are much closer to the targets when compared to the NARX Model 1. In Fig. 24 we see that by increasing the NARX's TDL size to 15 (NARX Model 2) we can match the same performance as the NARMAX Model 1.

6. Conclusion

The focus of this paper has been an in-depth comparison of NARX and NARMAX models for multi-step predictions. To the authors' knowledge, this is the first detailed comparison of NARX and NARMAX models when trained to optimize multi-step prediction errors.

Training NARMAX models for multi-step predictions necessitates several intricate procedures that are not required for the much more commonly used NARX models trained for one-step predictions. Because of the feedback in NARMAX models, dynamic backpropagation is required to compute exact gradients. In addition, the error surfaces of

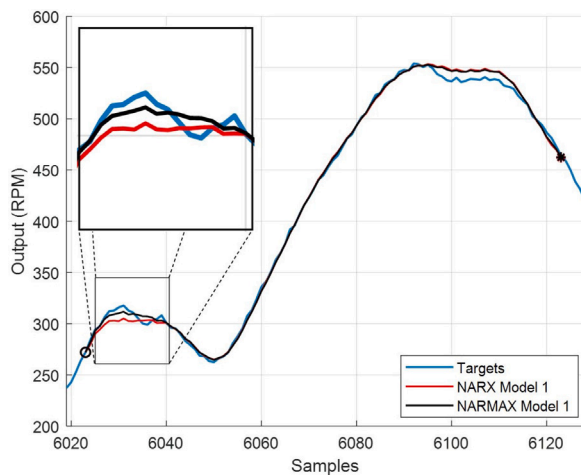


Fig. 23. NARX & NARMAX Model 1 – Trained on servo-motor data set for multi-step prediction for a maximum horizon of 100 – Circle marks the start of the prediction and star marks the end of the prediction.

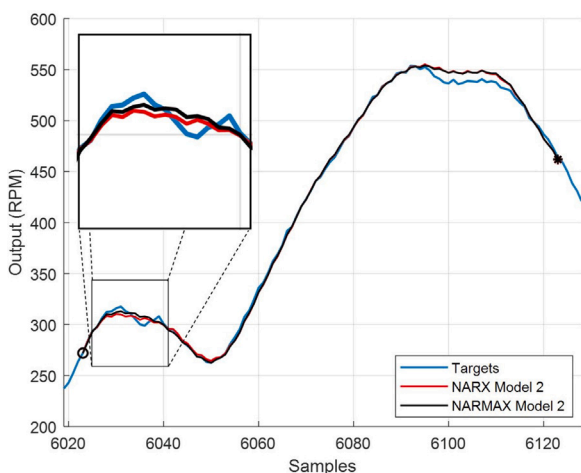


Fig. 24. NARX & NARMAX model 2 – Trained on servo-motor data set for multi-step prediction for a maximum horizon of 100 – Circle marks the start of the prediction and star marks the end of the prediction.

these recurrent networks often contain spurious valleys that impede standard training algorithms. Finally, there is a TDL loading problem that is much more difficult for NARMAX models. We have addressed these issues in this paper, some for the first time, and have successfully trained and deployed NARMAX multi-step predictors.

We have demonstrated on simulated data sets and also on two real-world data sets how NARMAX models can provide parsimonious and more accurate multi-step predictors than NARX models for certain types of data. While NARMAX training is somewhat more difficult than NARX training, the multi-step predictions can be more accurate, especially for short prediction horizons. For long prediction horizons there is no significant difference between the NARX and NARMAX predictions.

For some types of data sets the DGP has NARX characteristics, while for other data sets the DGP is NARMAX-like. The NARMAX model can more efficiently handle processes with complex disturbances that are not correlated with measured process inputs, as was the case for the servo-motor dataset investigated in this paper. The NARX model is more effective for DGP with small or simple disturbances, as in the maglev dataset considered here.

CRediT authorship contribution statement

Joseph Kelley: Conceptualization, Methodology, Software, Visualization, Writing – original draft, Formal analysis, Validation, Investigation. **Martin T. Hagan:** Conceptualization, Methodology, Writing – review & editing, Visualization, Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

References

- Acuña, G., Ramirez, C., & Curilem, M. (2012). Comparing NARX and NARMAX models using ANN and SVM for cash demand forecasting for ATM. In *The 2012 international joint conference on neural networks* (pp. 1–6). IEEE.
- Billings, S. A. (2013). *Nonlinear system identification: NARMAX methods in the time, frequency, and spatio-temporal domains*. John Wiley & Sons.
- Billings, S., & Leontaritis, I. (1982). Parameter estimation techniques for nonlinear systems. *IFAC Proceedings Volumes*, 15(4), 505–510.
- Chen, S., & Billings, S. A. (1989). Representations of non-linear systems: the NARMAX model. *International Journal of Control*, 49(3), 1013–1032.
- Chen, S., Billings, S. A., Cowan, C. F., & Grant, P. M. (1990). Practical identification of NARMAX models using radial basis functions. *International Journal of Control*, 52(6), 1327–1350.
- Cox, D. R. (1961). Prediction by exponentially weighted moving averages and related methods. *Journal of the Royal Statistical Society. Series B. Statistical Methodology*, 23(2), 414–422.
- De Jesus, O., & Hagan, M. T. (2007). Backpropagation algorithms for a broad class of dynamic networks. *IEEE Transactions on Neural Networks*, 18(1), 14–27.
- Gao, Y., & Er, M. J. (2005). NARMAX time series model prediction: feedforward and recurrent fuzzy neural network approaches. *Fuzzy Sets and Systems*, 150(2), 331–350.
- Golob, M., & Tovornik, B. (2008). Input-output modelling with decomposed neuro-fuzzy ARX model. *Neurocomputing*, 71(4–6), 875–884.
- Hagan, M. T., Demuth, H. B., & Jesús, O. D. (2002). An introduction to the use of neural networks in control systems. *International Journal of Robust and Nonlinear Control: IFAC-Affiliated Journal*, 12(11), 959–985.
- Hagan, M. T., & Menhaj, M. B. (1994). Training feedforward networks with the marquardt algorithm. *IEEE Transactions on Neural Networks*, 5(6), 989–993.
- Hedrea, E. L., Precup, R. E., Roman, R. C., & Petriu, E. M. (2021). Tensor product-based model transformation approach to tower crane systems modeling. *Asian Journal of Control*, 23(3), 1313–1323.
- Jafari, A. H., & Hagan, M. T. (2015). Enhanced recurrent network training. In *2015 international joint conference on neural networks* (pp. 1–8). IEEE.
- Jafari, A. H., & Hagan, M. T. (2018). Application of new training methods for neural model reference control. *Engineering Applications of Artificial Intelligence*, 74, 312–321.
- Jeong, K., Kahng, A. B., Lin, B., & Samadi, K. (2010). Accurate machine-learning-based on-chip router modeling. *IEEE Embedded Systems Letters*, 2(3), 62–66.
- Kelley, J., & Hagan, M. (2020). New fault diagnosis procedure and demonstration on hydraulic servo-motor for single faults. *IEEE/ASME Transactions on Mechatronics*, 25(3), 1499–1509.
- Ketsarapong, S., Punyngarm, V., Phusavat, K., & Lin, B. (2012). An experience-based system supporting inventory planning: A fuzzy approach. *Expert Systems with Applications*, 39(8), 6994–7003.
- Ljung, L. (1999). In Prentice Hall (Ed.), *System identification: Theory for the user, prentice hall information and system sciences series*. New Jersey.
- McHugh, C., Coleman, S., Kerr, D., & McGlynn, D. (2018). A linear polynomial NARMAX model with multiple factors to forecast day-ahead electricity prices. In *2018 IEEE symposium series on computational intelligence* (pp. 2125–2130). IEEE.
- Narendra, K. S., & Parthasarathy, K. (1990). Identification and control of dynamical systems using neural networks. *IEEE Transactions on Neural Networks*, 1(1), 4–27.
- Phan, M. C., & Hagan, M. T. (2013). Error surface of recurrent neural networks. *IEEE Transactions on Neural Networks and Learning Systems*, 24(11), 1709–1721.
- Pozna, C., Precup, R. E., Tar, J. K., Škrjanc, I., & Preitl, S. (2010). New results in modelling derived from Bayesian filtering. *Knowledge-Based Systems*, 23(2), 182–194.

- Precup, R. E., Duca, G., Travin, S., & Zinicovscaia, I. (2022). Processing, neural network-based modeling of biomonitoring studies data and validation on Republic of Moldova data. *Proceedings of the Romanian Academy, Series A: Mathematics, Physics, Technical Sciences, Information Science*, 23(4), 403–410.
- Tiao, G. C., & Tsay, R. S. (1994). Some advances in non-linear and adaptive modelling in time-series. *Journal of Forecasting*, 13(2), 109–131.
- Tiao, G. C., & Xu, D. (1993). Robustness of maximum likelihood estimates for multi-step predictions: the exponential smoothing case. *Biometrika*, 80(3), 623–641.
- Verma, A., Meenpal, T., & Acharya, B. (2022). Computational cost reduction of convolution neural networks by insignificant filter removal. *Science and Technology*, 25(2), 150–165.
- Wei, H., & Billings, S. (2006). Long term prediction of non-linear time series using multiresolution wavelet models. *International Journal of Control*, 79(06), 569–580.